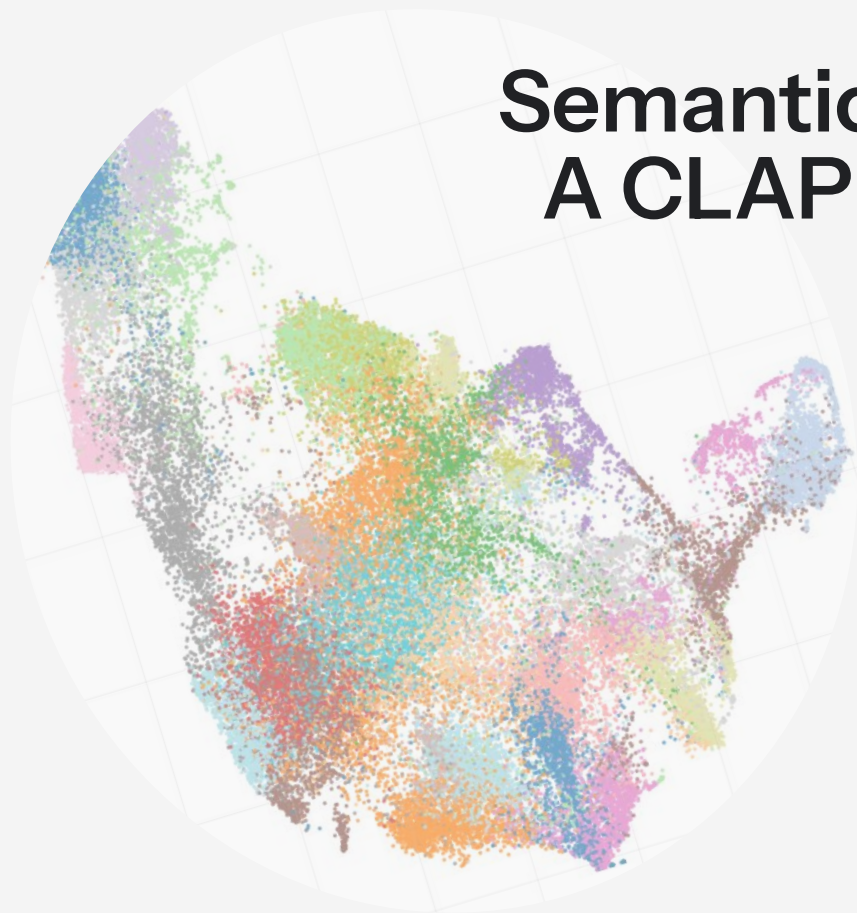


Semantic Overlap in Music Tags: A CLAP Embedding Analysis of MTG-Jamendo



Motivation

What are tags?

- Short, human labels (genre, mood, instrument, etc.) that describe how a track (song) sounds

How are tags used?

- Power search, playlist organization, and recommendation

What is tag polysemy?

- One tag used for multiple, sonically different regions (“ambient” can mean techno, piano, jazz, etc.; mood/theme tags are especially polysemous)



Why does polysemy matter?

- If a tag covers many distinct sounds, models may learn fuzzy labels, evaluations may become misleading, and recommendation/search results may become noisy or inaccurate

RQ + Project Goals

Research Question:

- What patterns of tag polysemy emerge in a music embedding space—do music tags (genre, instrument, mood) map to single, coherent regions, or do they split into multiple distinct clusters corresponding to different acoustic senses of the same tag?

Goals:

- Build an unsupervised clustering pipeline on song embeddings
- Visualize and examine cluster structure
- Inspect how tags occupy clusters (toward polysemy analysis)

Related Works

Auto-tagging on MTG-Jamendo:

- Learning models trained to predict tag labels directly from audio, optimizing for tagging metrics but not analyzing the structure of the embedding space

Projecting Latent Representations of Audio:

- Audio encoded and projected onto 2D space to analyze musical similarities between songs

Music Information Retrieval (MIR):

- Studied noisy/user-generated tags and tag co-occurrence graphs for global statistics but not in the embedding space

Novelty:

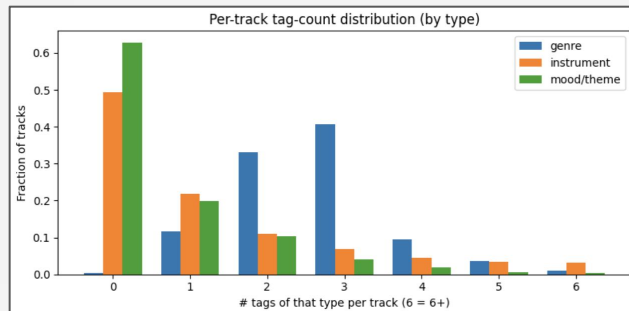
- Characterizing tag polysemy in audio space using tools from related works

Background: MTG-Jamendo Dataset

What is MTG-Jamendo?

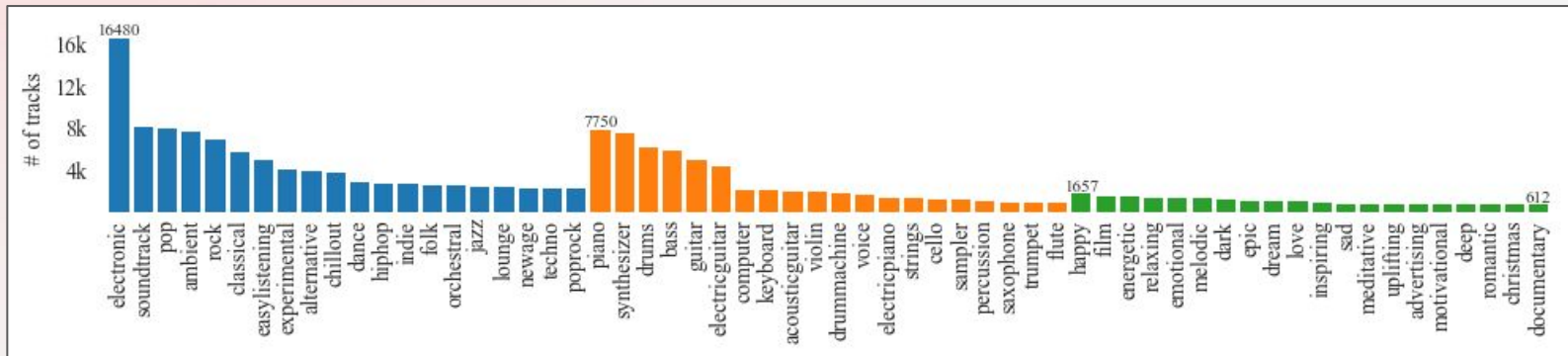
- Open music dataset tracks from Jamendo (indie artists, royalty-free music)
- ~55.6k tracks (156GB) after filtering
- Multiple tag families per track:

- ◆ **Genre** (e.g., rock, techno, hip-hop)
- ◆ **Instrument** (e.g., guitar, synth, violin)
- ◆ **Mood/theme** (e.g., happy, film, dark)



Why this dataset?

- Openly licensed, designed for auto-tagging with multi-label tag metadata, large/diverse enough for clustering and tag-pattern analysis



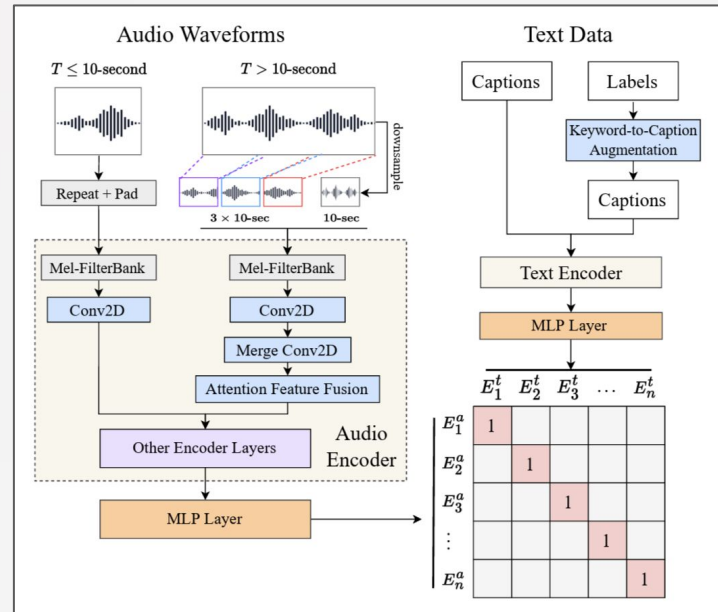
Background: CLAP + Embedding Space

What is CLAP?

- **Contrastive Language–Audio Pretraining model:** audio + text encoders map both into shared embedding space
- For this project: CLAP as *audio-only* feature extractor:
 - ◆ output = 512-D vector per track

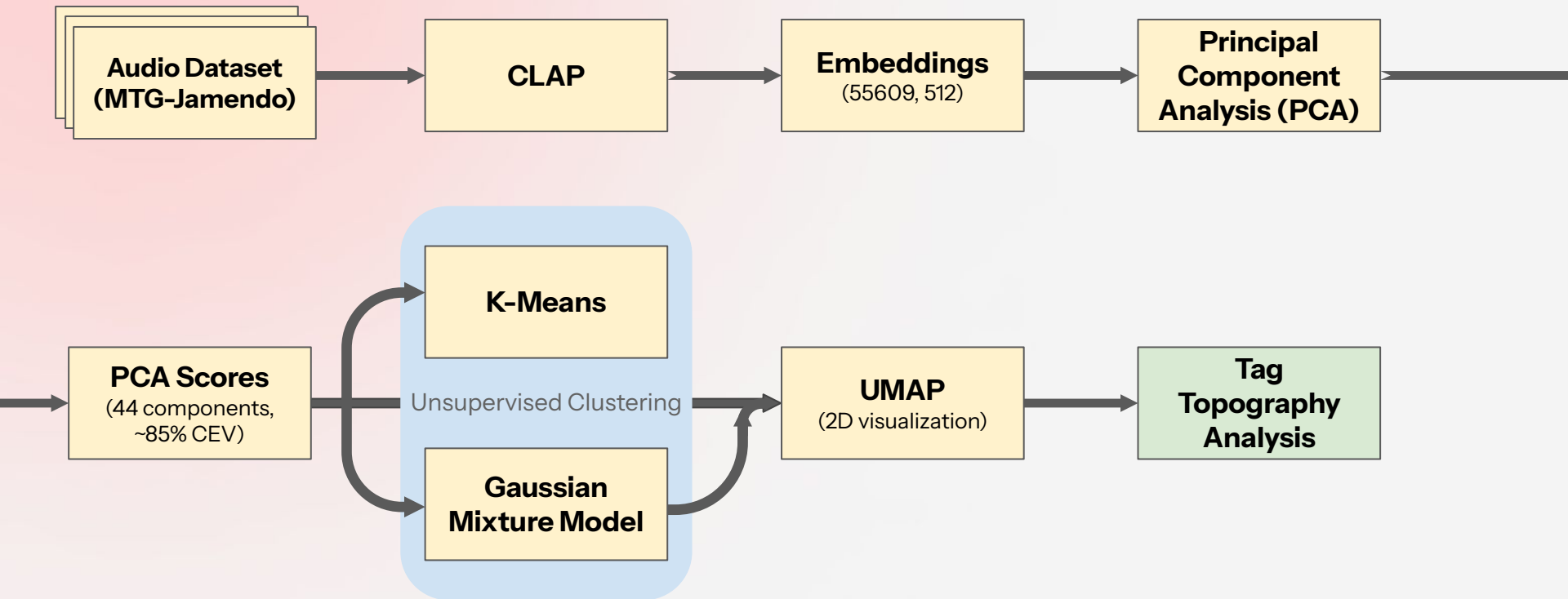
Practical setup

- CLAP is trained on audio-text pairs:
 - ◆ music embeddings are “language-shaped”
- 156GB MTG-Jamendo audio on Google Drive in small batches with the *music_audioset_epoch_15_esc_90.14.pt* checkpoint, saving one parquet file (path, track_id, embedding[512])



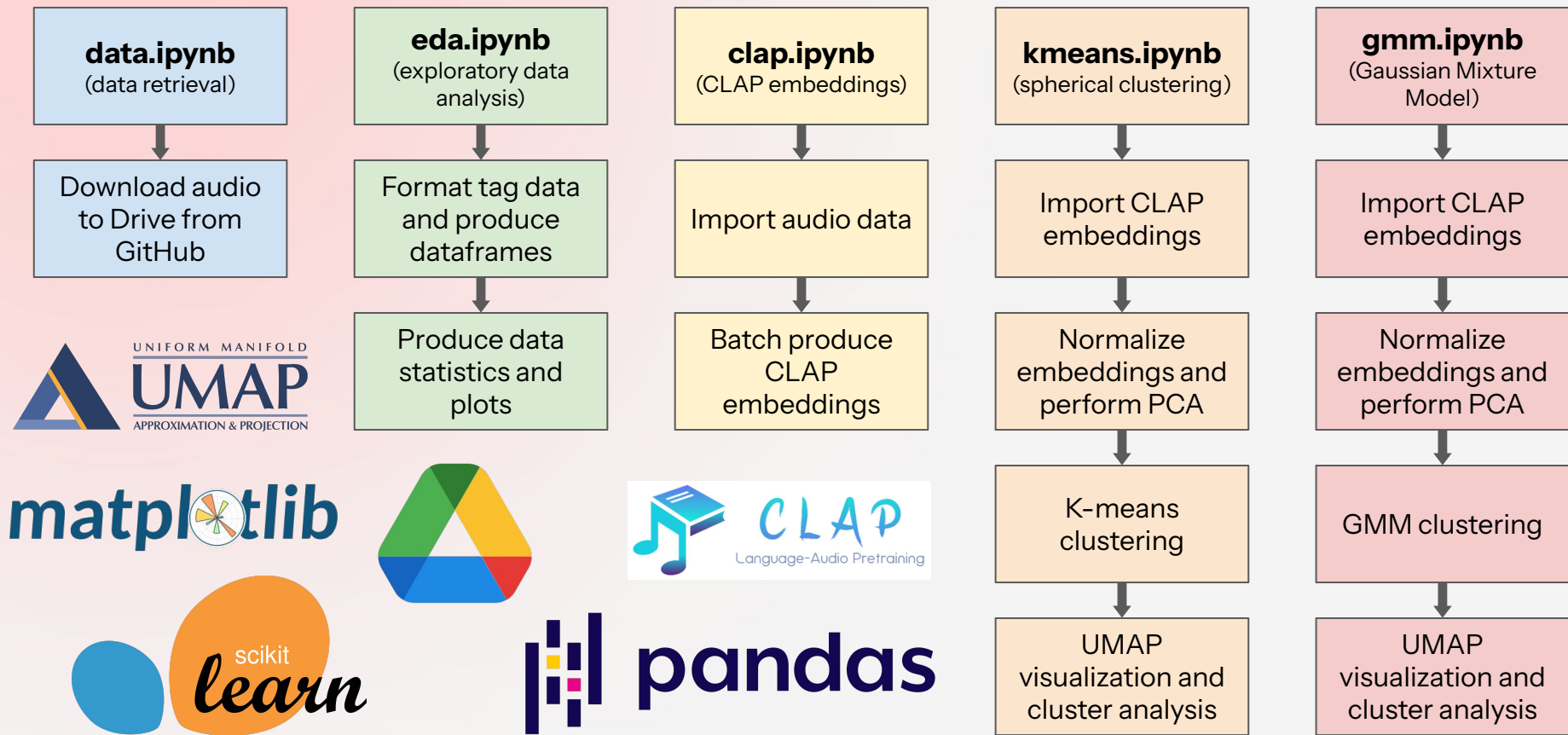
Pipeline Overview

Date



Implementation Overview

Date



PCA: Dimensionality Reduction

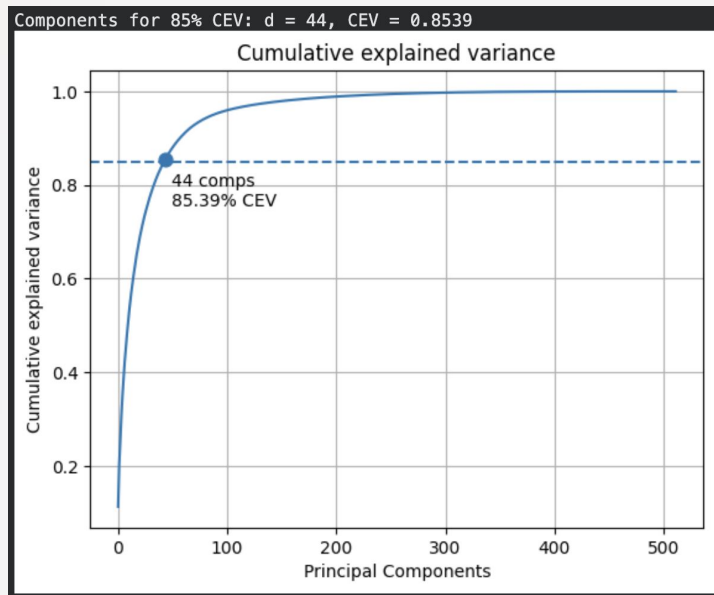
Date

Why reduce CLAP's 512-D space?

- PCA makes distances more meaningful by removing low-variance “noise” dimensions
- keeps GMM covariance estimation numerically stable and computationally feasible

How many components?

- PCA fit on the L2-normalized embeddings and plotted cumulative explained variance:
 - ◆ 44 principal components capture ~85% of the variance



Attempt #1: K-Means Clustering

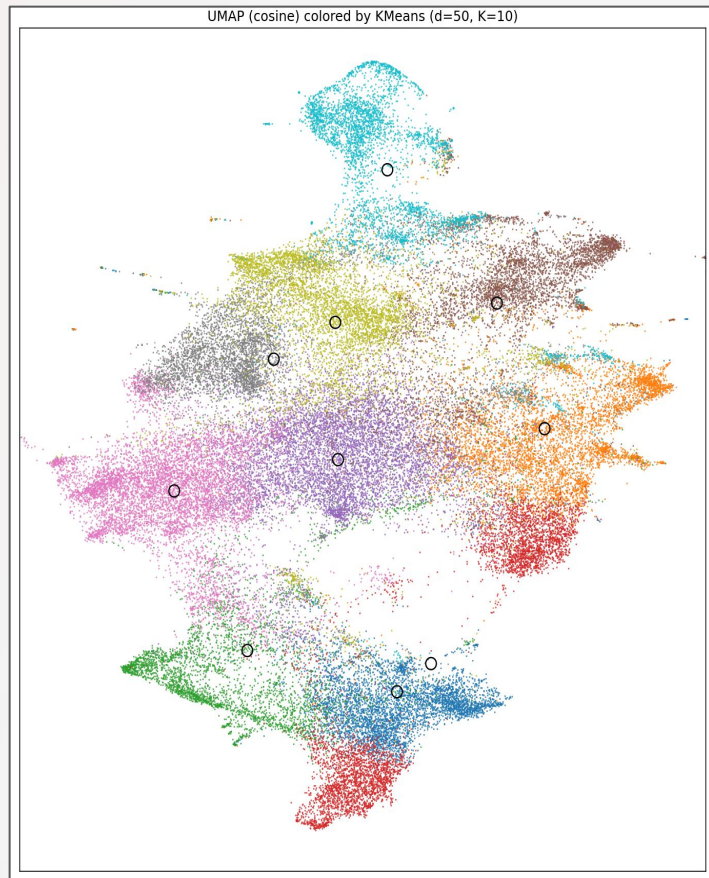
Date

Setup

- K-means sweep on 50-D PCA scores, with K ranging [1, 30)

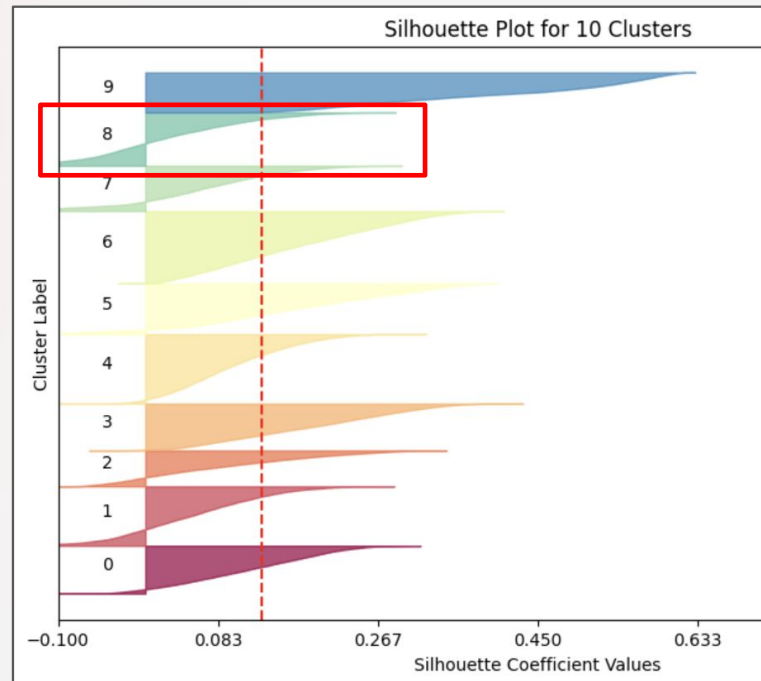
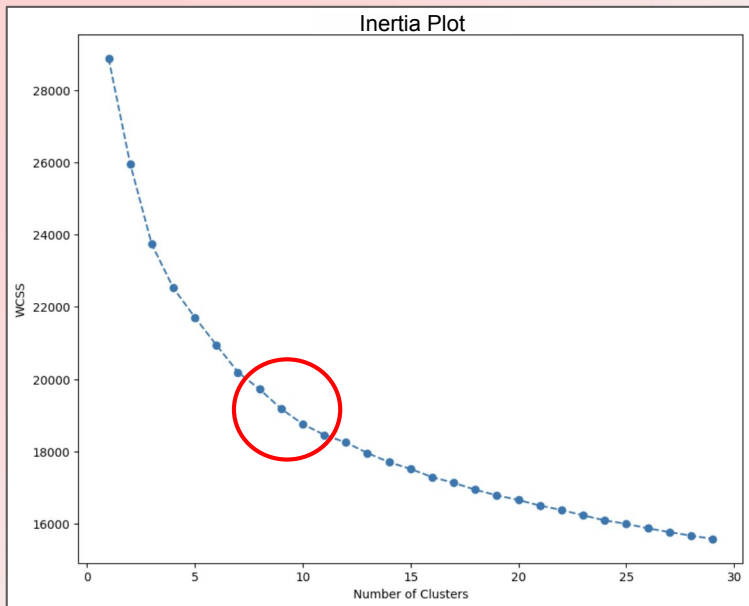
Metrics computed

- Tracked inertia (within-cluster SSE, elbow method) for cluster compactness and silhouette scores for song-to-cluster fit



Attempt #1: K-Means Clustering

Date



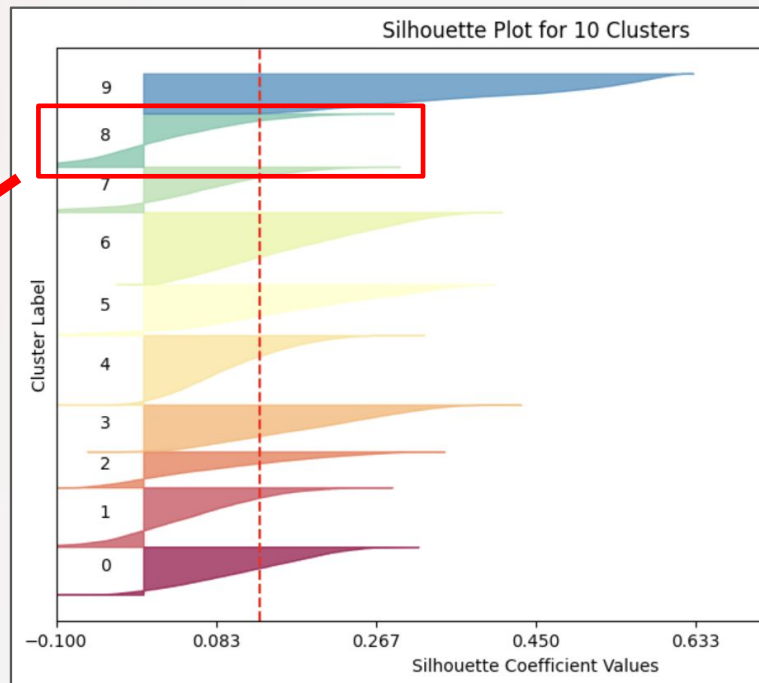
No clear elbow in inertia; silhouette scores stayed low across K , with many points showing negative silhouettes (poor separation)...

Attempt #1: K-Means Clustering

Date

Worst tags in Cluster 8 (by (-) silhouette):

	cluster	tag	n_tracks_neg
1223	8	genre---electronic	976
1198	8	genre---ambient	908
1316	8	instrument---synthesizer	481
1273	8	genre---soundtrack	472
1229	8	genre---experimental	404
1310	8	instrument---piano	271
1221	8	genre---easylistening	268
1255	8	genre---newage	233
1205	8	genre---chillout	213
1207	8	genre---classical	191



Takeaway

- K-means not capturing clean, spherical clusters, suggesting overlapping structure... switch to a more flexible probabilistic model (GMM)

Attempt #2: Gaussian Mixture Model

What is a GMM?

- Probabilistic mixture of Gaussians: each cluster has its own mean (centroid) and covariance (shape/orientation), and each track gets a probability for each cluster

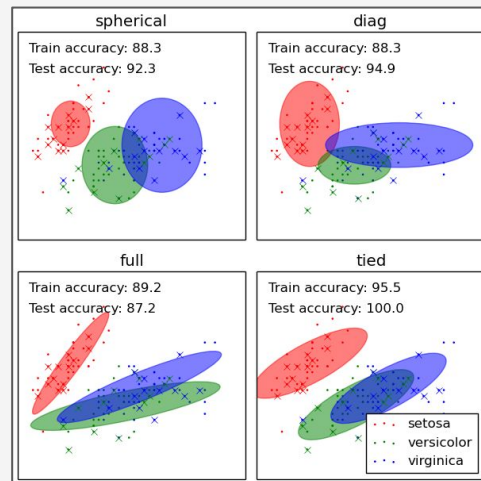
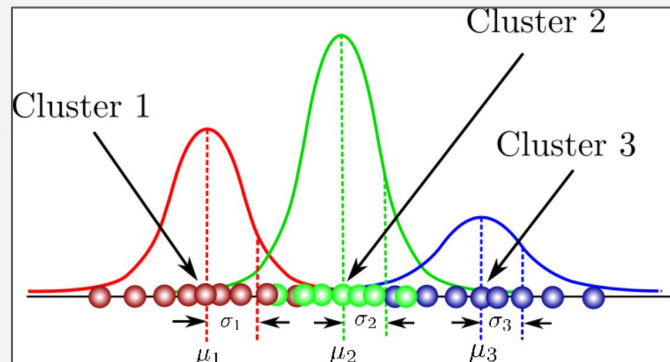
Model variants

- Covariance types: {spherical, tied, diagonal, and full}

Grid search setup

- Fit GMMs on the 44-D PCA scores for K from 1 to 31 and all four covariance types, comparing models using AIC and BIC (Akaike + Bayesian Information Criterion)

(Long sweep → saved metrics to .npz on Drive and stored the best-fitted GMM with joblib)



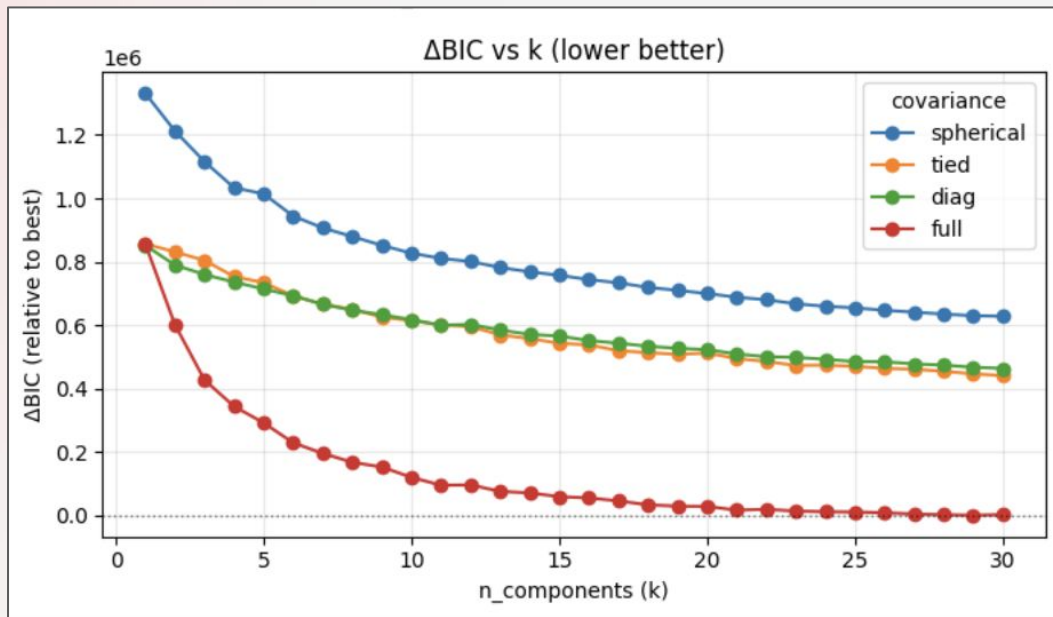
GMM Selection: AIC/BIC, Heatmaps

AIC/BIC for model choice

- AIC and BIC balance fit vs. complexity:
 - ◆ lower scores = well-fit without too many parameters

ΔBIC plots

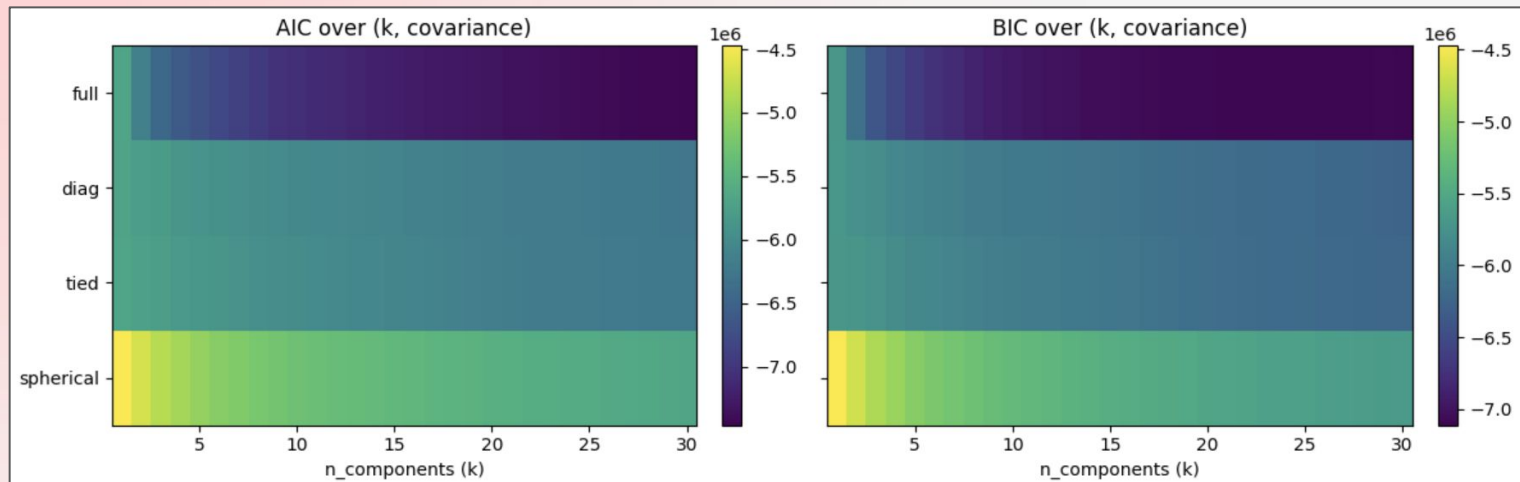
- ΔBIC curves show how far each model is from the global best; the winning model's line approaches 0 (best - best = 0)



GMM Selection: AIC/BIC, Heatmaps

Heatmaps

→ AIC/BIC heatmaps over K and covariance type show full covariance consistently lowest



Chosen model

- GMM with full covariance at K = 15
- ◆ allows flexible, arbitrarily oriented ellipsoidal clusters in PCA space (unlike K-means' strictly spherical clusters)

UMAP (Uniform Manifold Approximation and Projection)

UMAP for visualization only

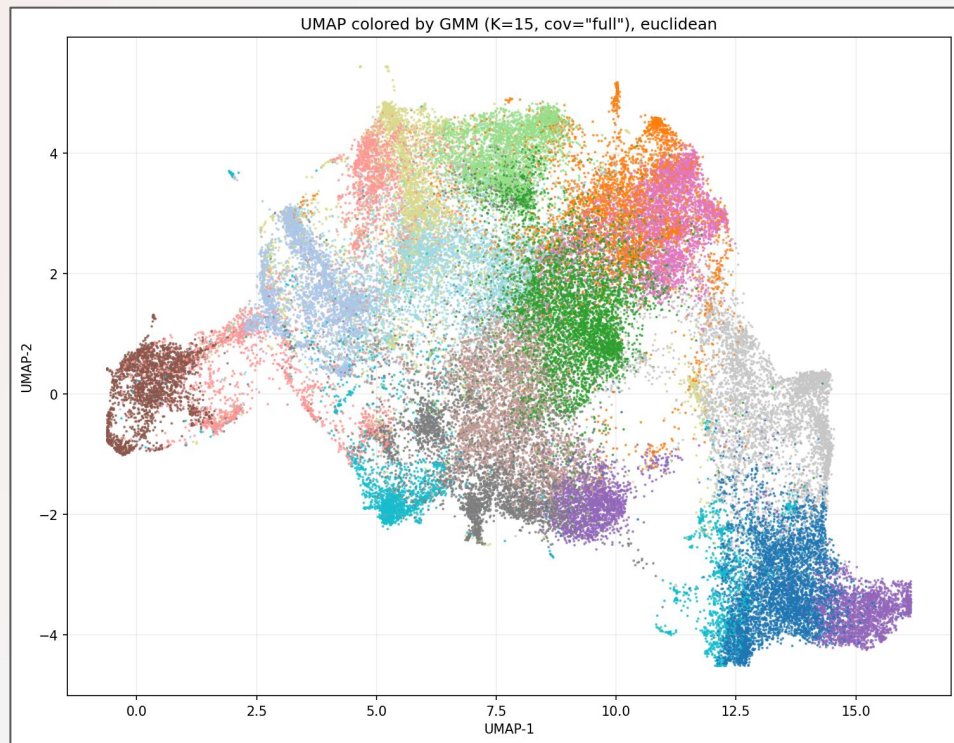
- UMAP (second nonlinear DR) on 44-D PCA scores to get a 2D embedding that preserves local neighborhood structure

Handling outliers

- I clip the view to the 1st–99th percentiles so extreme outliers don't dominate or distort the plot

What the plot shows

- Each point is a song, colored by its GMM component
- we see distinct but overlapping “islands” of clusters that reveal the learned structure in CLAP space

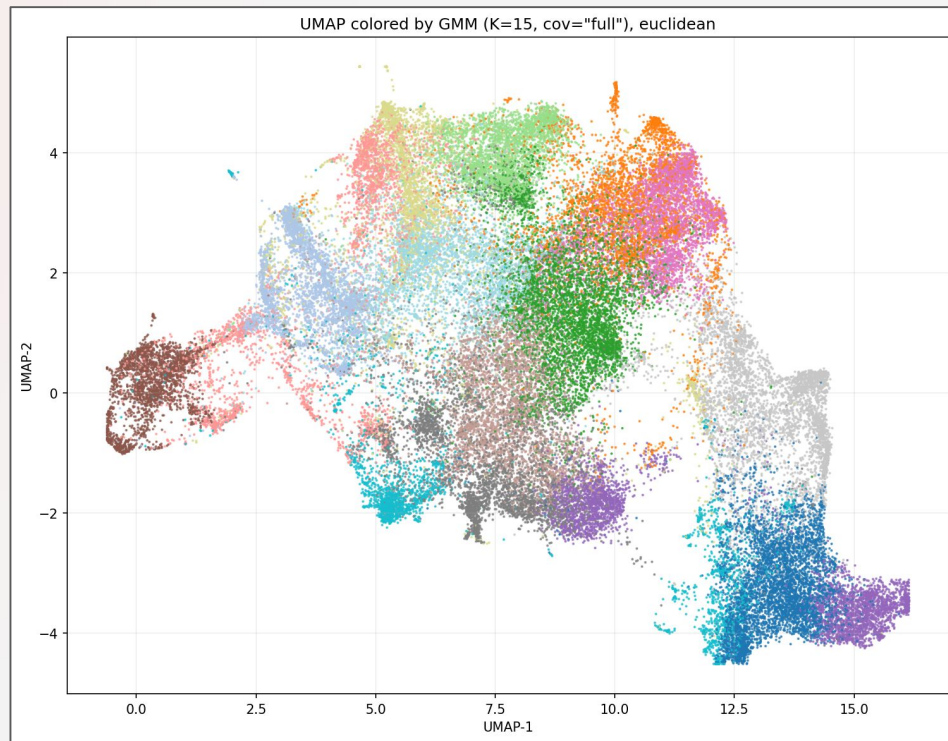


Recap: K-Means to GMM

UMAP (cosine) colored by KMeans (d=50, K=10)



UMAP colored by GMM (K=15, cov="full"), euclidean



Tag Integration

Joining tags to embeddings

- Joined the CLAP/PCA/GMM/UMAP table with the MTG-Jamendo autotagging file on track_id

	TRACK_ID	ARTIST_ID	ALBUM_ID	PATH	DURATION	TAGS
0	track_0000214	artist_000014	album_000031	14/214.mp3	124.6	genre---punkrock
1	track_0000215	artist_000014	album_000031	15/215.mp3	151.4	genre---metal
2	track_0000216	artist_000014	album_000031	16/216.mp3	234.9	genre---metal
3	track_0000217	artist_000014	album_000031	17/217.mp3	127.9	genre---punkrock
4	track_0000218	artist_000014	album_000031	18/218.mp3	180.7	genre---punkrock
5	track_0000219	artist_000014	album_000031	19/219.mp3	199.4	genre---metal
6	track_0000220	artist_000014	album_000031	20/220.mp3	174.3	genre---punkrock
7	track_0000221	artist_000014	album_000031	21/221.mp3	224.6	genre---punkrock
8	track_0000222	artist_000014	album_000031	22/222.mp3	161.9	genre---punkrock
9	track_0000223	artist_000014	album_000031	23/223.mp3	217.9	genre---metal

autotagging table

Cleaning and mismatches

- Dropped songs with missing tag info and removed redundant columns (extra IDs, file paths)

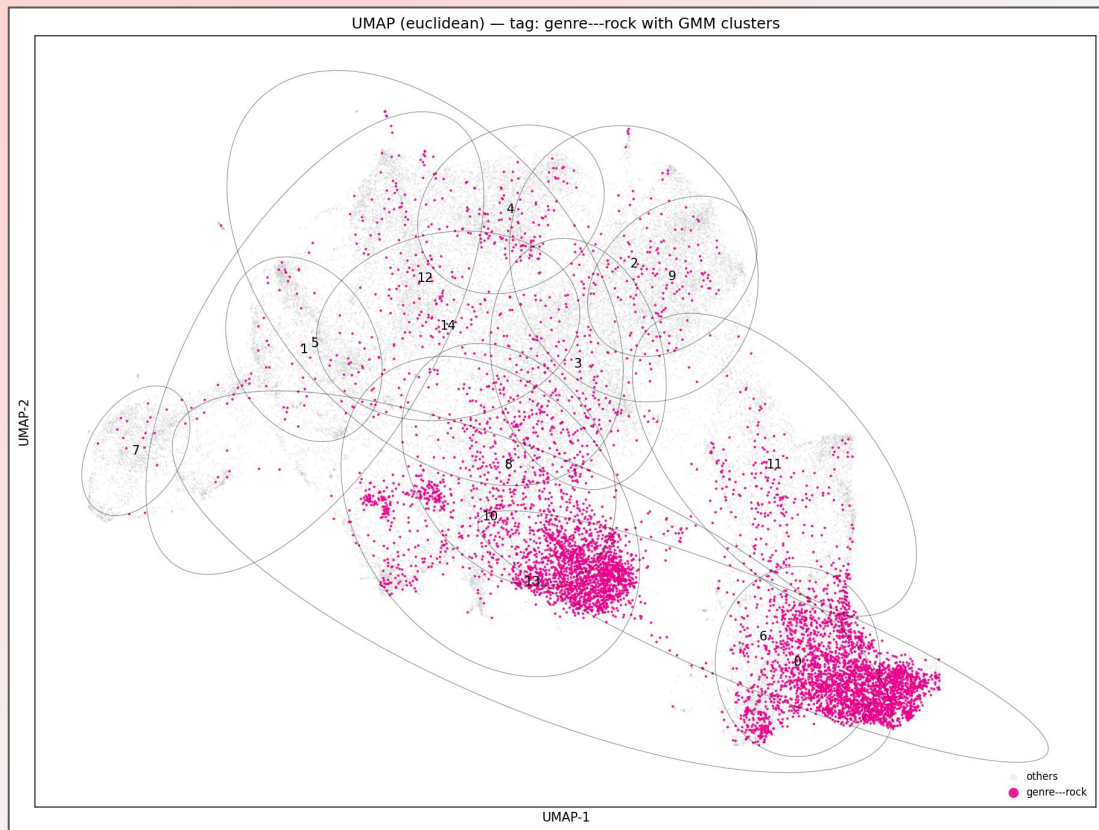
Tag normalization

- Built a tags_list column: a Python list of all tags per track (across families)
 - ◆ helps quickly query tag presence and visualize tag topography on the map

	track_id	cluster	umap_x	umap_y	tags_list
0	1100507	11	13.097302	-0.014914	[genre---ethnicrock, genre---hiphop, genre---p...
1	1129026	13	5.124250	-1.726381	[genre---classical, instrument---classicalguit...
2	710586	8	7.800599	-0.579605	[genre---acidjazz, genre---ambient, genre---ch...
3	492112	11	12.502379	-0.175134	[genre---electronic]
4	1318107	12	5.234650	1.201381	[genre---classical, genre---orchestral, genre---

New table with cluster assignments AND tags

Map Analysis...

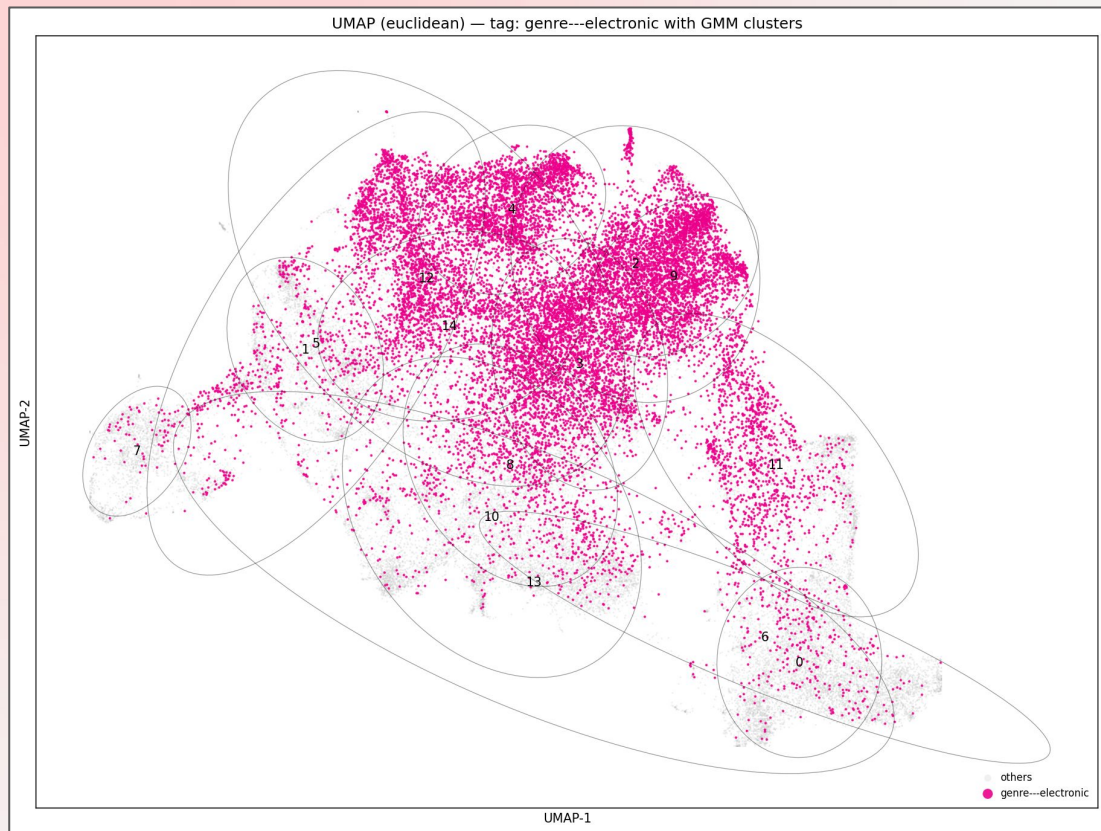


“Clustered” tags:

- Dense islands, low overlap
- Relatively coherent acoustic region
- Low-apparent polysemy?

Ex. genre-rock

Map Analysis...



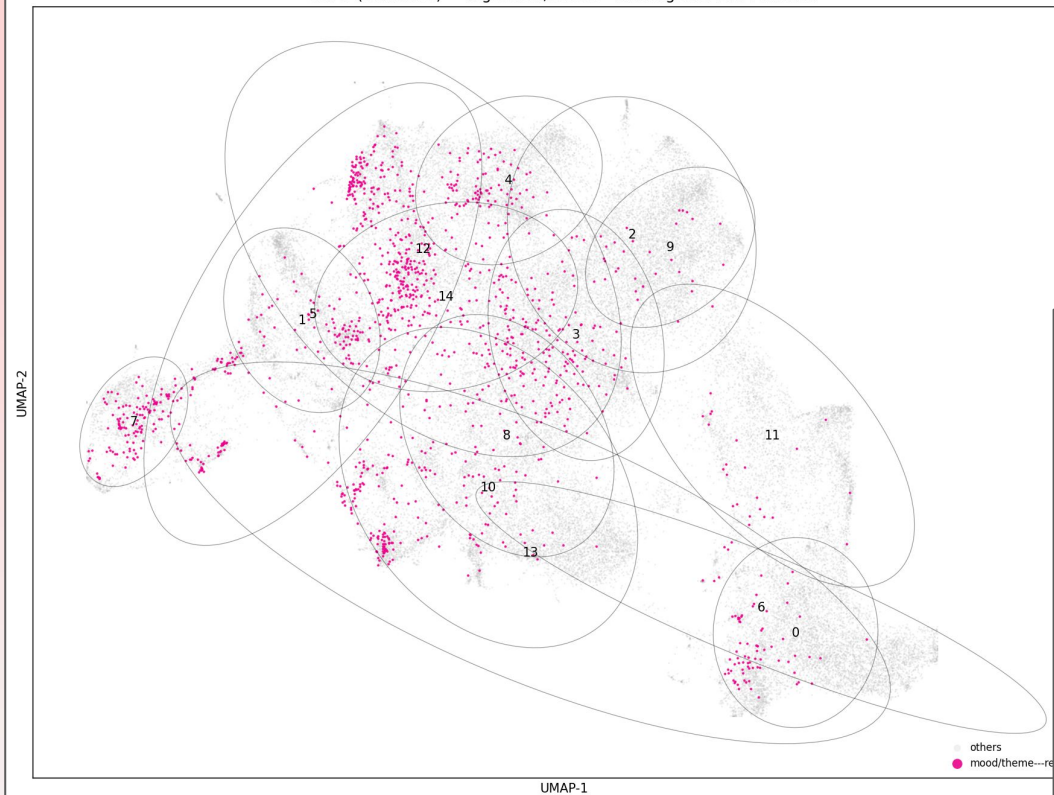
“Scattered” tags:

- Multiple islands
- High overlap
- High-apparent polysemy?

Ex. genre–electronic

Map Analysis...

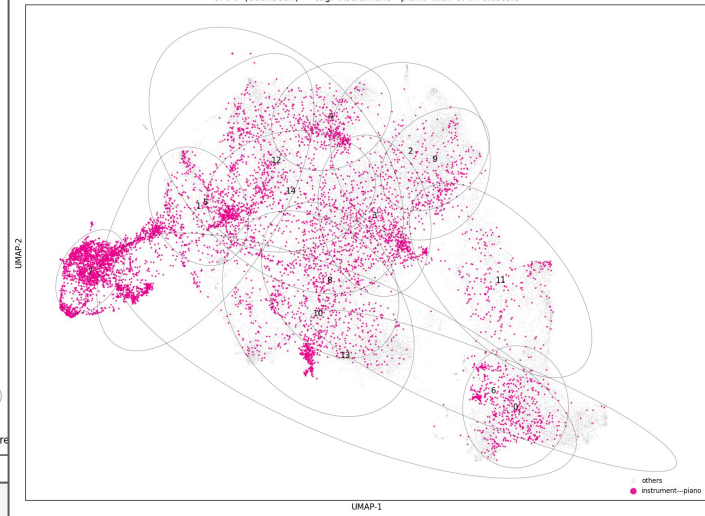
UMAP (euclidean) — tag: mood/theme---relaxing with GMM clusters



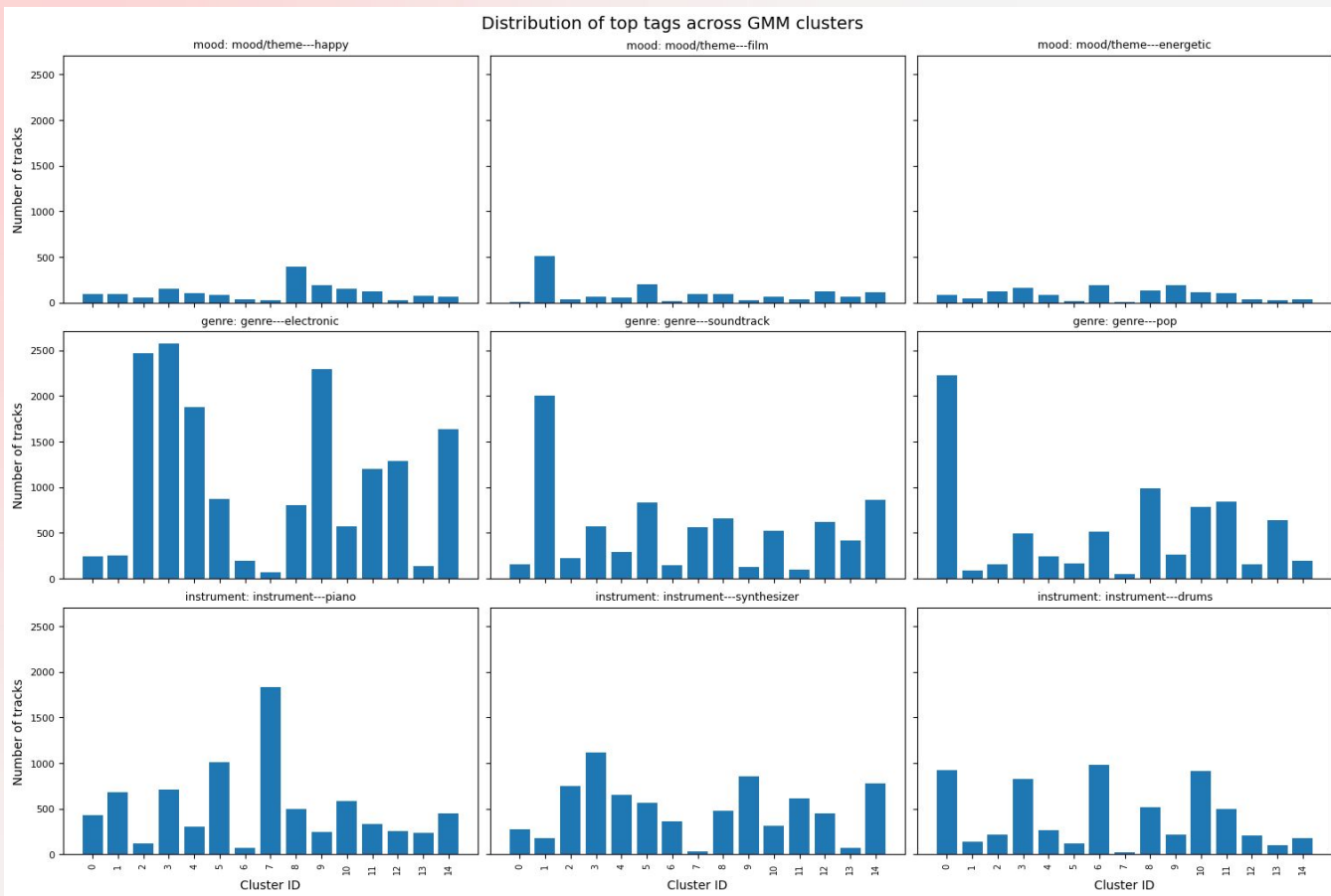
“Sparse” tags:

- Low volume
- Multiple regions, some distinct
- High polysemy?

UMAP (euclidean) — tag: instrument---piano with GMM clusters



Quantifying Tag-Cluster Relationships



Quantifying Tag-Cluster Relationships

Least polysemous:

- Low entropy
- High top1_mass (fraction of tags in most common cluster)

Tags are in the most common cluster and have low spread

	tag	n_tracks	n_clusters	entropy	entropy_norm	top1_mass	top3_mass
	genre---hardrock	490.0	12.0	1.098284	0.441982	0.712245	0.875510
	genre---rap	993.0	15.0	1.273932	0.470424	0.646526	0.869084
	genre---punkrock	729.0	13.0	1.233492	0.480903	0.668038	0.879287
	genre---singersongwriter	776.0	15.0	1.372554	0.506842	0.563144	0.864691
	genre---grunge	375.0	11.0	1.222890	0.509985	0.624000	0.904000

Most polysemous:

- High entropy
- Low top1_mass (fraction of tags in most common cluster)

Tags are spread across components and do not align to any one cluster

	tag	n_tracks	n_clusters	entropy	entropy_norm	top1_mass	top3_mass
	mood/theme---melodic	1213.0	15.0	2.592908	0.957482	0.114592	0.317395
	mood/theme---dream	951.0	15.0	2.565382	0.947317	0.143007	0.357518
	mood/theme---sad	749.0	15.0	2.538371	0.937343	0.133511	0.368491
	genre---psychedelic	590.0	15.0	2.532252	0.935083	0.127119	0.349153
	mood/theme---melancholic	441.0	15.0	2.531720	0.934887	0.131519	0.358277

Future Work

Audio-text alignment

- Use CLAP's text encoder and compare them to audio cluster centroids

Richer map analyses

- Examine clusters themselves (top tags + audio examples) to characterize what acoustic dimensions the GMM is picking up (tempo, instrumentation, production style, etc.)

Downstream implications

- Investigate whether splitting highly polysemous tags into data-driven sub-tags improves supervised taggers or recommendation quality

Thank you!

